LP - III

Mini Project Report

On

Titanic Survival Prediction Using Machine Learning

Submitted in the fulfillment of requirements of Savitribai Phule Pune University

For the degree of **Third Year of Computer Engineering**

AY 2026-27

By

Mayur Zope TECA171

Yash Chaudhary TECA170

Prem Borde TECA111

Under the guidance of

Prof. Akansha Damonkar



Department of Computer Engineering

PCET-NMVPM's

NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING AND TECHNOLOGY

TALEGAON-DABHADE,

Vishnupuri, Talegaon Station-410507

SAVITRIBAI PHULE PUNE UNIVERSITY



NUTAN MAHARASHTRA INSTITUTE OF ENGINEERING & TECHNOLOGY, TALEGAON-DABHADE

CERTIFICATE

This is to Certify that -

Name	PRN
Mayur Zope	72322283F
Yash Chaudhary	72322276C
Prem Borde	72321957F

Has satisfactorily completed the requirements of Project under Machine Learning Mini Project

For the degree of **Computer Engineering**

On –

Titanic Survival Prediction Using Machine Learning

Prof. Akansha Damonkar	Dr. Rahul Patil	Dr. Pramod Patil
Course Coordinator	Head of Department	Director

Acknowledgement

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individuals and organizations. We would like to extend our sincere thanks to all of them.

We are highly indebted to Prof. Akansha Damonkar for her guidance and constant supervision, as well as for providing the necessary information regarding the project and for her support in completing it.

We would like to thank our HOD, Dr. Rahul Patil, for his constant support in all project-related activities.

We would like to express our gratitude towards our parents and the members of NMIET for their kind co-operation and encouragement, which helped us in the completion of this project.

Our thanks and appreciation also go to our colleagues who helped us in developing the project and to everyone who willingly helped us with their abilities.

Abstract

Machine learning has become a practical way to study historical records and to predict an outcome that is not known in advance. This project presents the development of a classification system titled Titanic Survival Prediction Using Machine Learning. The work uses the Seaborn Titanic dataset of 891 passengers and builds models that predict whether a passenger survived the disaster.

The main objective of the application is to convert raw passenger attributes into a clean feature set and to compare supervised learning algorithms on that set. Exploratory data analysis is first performed to study survival with respect to class, gender, age, fare, family size and port of embarkation. Missing values in age, deck, embarked and embark_town are treated, categorical fields are encoded, and a family_size feature is created. The data is then split into training and testing sets in an 80:20 ratio.

Two classifiers are trained: Logistic Regression and Random Forest. Logistic Regression achieved a test accuracy of 81.56 percent. A default Random Forest scored 78.21 percent, and after GridSearchCV tuning of n_estimators, max_depth and min_samples_split the optimized Random Forest reached 81.01 percent. Precision, recall, F1-score and confusion matrices are reported for each model. This project demonstrates the practical pipeline of exploratory analysis, preprocessing, model training, evaluation and hyperparameter optimization.

Keywords: Machine Learning, Titanic Dataset, Classification, Logistic Regression, Random Forest, Exploratory Data Analysis, Scikit-learn, GridSearchCV

CONTENTS

Chapter	Title
I	Certificate
II	Acknowledgement
III	Abstract
1	Problem Statement
2	Software and Hardware Requirements
3	Proposed Methodology
4	Design & Theory
5	Code & Output
6	Conclusion
7	References

CHAPTER 1: PROBLEM STATEMENT

In many real-world situations, an outcome of interest is binary: a passenger either survived or did not survive, a customer either leaves or stays, a transaction is either genuine or fraudulent. Traditional inspection of tables cannot capture the combined effect of several attributes at once. Manual rules also fail when the data contains missing values, mixed data types and overlapping patterns.

The major problems associated with analysing a historical disaster dataset such as Titanic include:

- Missing and noisy attributes: Age is missing for 177 passengers and deck is missing for 688 passengers, so a model cannot be trained on the raw table.
- Mixed data types: The table contains integers, floating-point values, categories and Boolean flags that must be converted into numeric features.
- Uneven survival rates: Survival is strongly associated with class and gender, and a naive guess that nobody survived would still be correct for about 62 percent of rows.
- Need for a fair test: A model that only memorises the training rows is of no use. A held-out test set is required to measure true predictive ability.
- Need to compare algorithms: A single model does not show whether a linear method or an ensemble method is more suitable for this data.

With the growing use of data-driven systems, there is a strong requirement for a complete machine learning pipeline that can clean the data, extract useful features, train classifiers and report accuracy with precision and recall.

Therefore, this project proposes the development of a Titanic Survival Prediction system using Python, Pandas, Seaborn and Scikit-learn. The system aims to predict passenger survival from ticket class, sex, age, fare, family information and embarkation details, and to compare Logistic Regression with Random Forest before and after hyperparameter tuning.

CHAPTER 2: SOFTWARE / HARDWARE REQUIREMENTS

2.1 Software Requirements

Component	Details
Programming Language	Python 3
Development Environment	Jupyter Notebook / Kaggle Notebook
Data Handling	NumPy, Pandas
Visualization	Matplotlib, Seaborn
Machine Learning Library	Scikit-learn
Dataset	Seaborn Titanic dataset (891 records, 15 columns)
Models	LogisticRegression, RandomForestClassifier, GridSearchCV
OS	Windows / Ubuntu / Kaggle Docker image
Browser	Google Chrome (for Kaggle / Jupyter)

2.2 Hardware Requirements

Component	Specification
Processor	Intel i3 or above
RAM	Minimum 4 GB
Storage	500 MB free space

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Methodology Overview

The project follows a structured machine learning approach starting from dataset loading and exploratory analysis to preprocessing, model training, evaluation and hyperparameter optimization. Survival is treated as a binary classification problem with the target column survived.

3.2 Steps Followed

- Load the dataset: The Seaborn Titanic dataset is loaded using sns.load_dataset('titanic'). It contains 891 rows and 15 columns.
- Inspect structure and quality: df.info(), df.head(), df.describe() and df.isnull().sum() are used to study data types, sample rows, summary statistics and missing values.
- Exploratory data analysis: Count plots, bar plots, histograms and box plots are drawn for survived, pclass, sex, age, fare, sibsp, parch, embarked, alone and deck.
- Handle missing values: Age is filled with the median within each pclass and sex group. Deck missing values are labelled Unknown. Embarked and embark_town are filled with the mode.
- Encode categorical variables: Sex is mapped to 0 and 1. Columns embarked, class, who, deck and embark_town are one-hot encoded with drop_first=True.
- Feature engineering: family_size is created as sibsp + parch. Irrelevant columns alive, adult_male and alone are dropped.
- Train-test split: Features X and label y are split with test_size=0.2 and random_state=42, giving 712 training rows and 179 test rows.
- Train baseline models: Logistic Regression and Random Forest are fitted on the training set.
- Evaluate models: Accuracy, classification report and confusion matrix are computed on the test set.
- Tune Random Forest: GridSearchCV searches n_estimators, max_depth and min_samples_split with 3-fold cross-validation. The best estimator is retrained and re-evaluated.

CHAPTER 4: DESIGN & THEORY

4.1 System Architecture

The system follows a conventional supervised learning architecture in which historical labelled examples are used to train a classifier that can label new passengers.

Architecture Components:

- Data Source: Seaborn Titanic dataset
- Analysis Layer: Pandas profiling, Seaborn and Matplotlib plots
- Preprocessing Layer: Imputation, encoding, feature creation and column dropping
- Modelling Layer: Logistic Regression and Random Forest
- Tuning Layer: GridSearchCV with 3-fold cross-validation
- Evaluation Layer: accuracy_score, classification_report and confusion_matrix

Working: The user runs the notebook cells in order. After cleaning, the feature matrix X and the target y are created. The training portion is used to fit each classifier. The testing portion is used only for evaluation. GridSearchCV internally splits the training data further in order to choose hyperparameters without looking at the test set.

4.2 Theory

- Supervised Learning: The algorithm is given examples that already contain the correct label, here survived = 0 or 1.
- Classification: The output is a discrete class rather than a continuous number.
- Logistic Regression: A linear model that estimates the probability of survival with a sigmoid function and a decision threshold of 0.5.
- Random Forest: An ensemble of decision trees. Each tree is trained on a bootstrap sample and a random subset of features. The forest votes to produce the final class.
- Train-Test Split: A portion of the data is held back so that reported accuracy reflects unseen passengers.
- Confusion Matrix: Counts of true negatives, false positives, false negatives and true positives.
- Precision: Of the passengers predicted to survive, the fraction who actually survived.
- Recall: Of the passengers who actually survived, the fraction that the model found.

- F1-score: The harmonic mean of precision and recall.
- GridSearchCV: Exhaustive search over a grid of hyperparameters using cross-validation.

4.3 Feature Design

Element	Type	Purpose
survived	int64 (target)	1 if the passenger survived, else 0
pclass / class_*	int / dummy	Ticket class; first class had higher survival
sex	encoded 0/1	Gender; female passengers survived more often
age	float, imputed	Age in years, filled by pclass and sex median
sibsp, parch, family_size	int	Family travelling together
fare	float	Ticket fare, related to class and survival
embarked_* / embark_town_*	dummy	Port of embarkation
who_*	dummy	Person type: man, woman or child
deck_*	dummy	Cabin deck, with Unknown for missing decks

CHAPTER 5: CODE & OUTPUT

5.1 Dataset Loading and Inspection

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

warnings.filterwarnings("ignore", category=UserWarning)



df = sns.load_dataset('titanic')

df.info()

print(df.head())

print(df.isnull().sum())

print(df.describe())
```

The dataset has 891 entries and 15 columns. Non-null counts show complete values for survived, pclass, sex, sibsp, parch and fare. Age has 714 non-null values, embarked and embark_town have 889, and deck has only 203. The mean of survived is 0.3838, so roughly 38 percent of passengers survived.

5.2 Exploratory Data Analysis

```
sns.countplot(x='survived', data=df)

plt.title('Survival Count')

plt.show()



sns.countplot(x='pclass', data=df)

plt.title('Passenger Class Distribution')

plt.show()
```

```
sns.barplot(x='pclass', y='survived', data=df)

plt.title('Survival Rate by Passenger Class')

plt.show()



sns.countplot(x='sex', data=df)

plt.title('Gender Distribution')

plt.show()



sns.barplot(x='sex', y='survived', data=df)

plt.title('Survival Rate by Gender')

plt.show()



df['age'].plot(kind='hist', bins=30, color='lightblue')

plt.title('Age Distribution')

plt.show()



sns.boxplot(x='survived', y='age', data=df)

plt.title('Age vs Survival')

plt.show()



df['fare'].plot(kind='hist', bins=30, color='lightgreen')

plt.title('Fare Distribution')

plt.show()



sns.boxplot(x='survived', y='fare', data=df)

plt.title('Fare vs Survival')
```

```
plt.show()
```

Further plots were drawn for sibsp, parch, embarked, alone and deck, together with the corresponding survival-rate bar plots. The visual study supports the following observations.

Figure	Observation
Survival Count	More passengers died than survived.
Passenger Class Distribution	Third class is the largest group.
Survival Rate by Passenger Class	First class has the highest survival rate.
Gender Distribution	There are more male passengers than female passengers.
Survival Rate by Gender	Female survival rate is much higher than male survival rate.
Age Distribution	Most passengers are between about 20 and 40 years.
Age vs Survival	Age overlap is large; age alone is a weak separator.
Fare Distribution	Fares are right-skewed, with a few very expensive tickets.
Fare vs Survival	Survivors tend to have paid a higher fare.
Family features	Travelling with a small family is associated with better survival than travelling alone or with a very large family.
Embarkation Port	Most passengers boarded at Southampton (S); Cherbourg (C) shows a higher survival rate.
Deck	Known cabin decks have higher plotted survival, but most values are missing.

5.3 Data Preprocessing

```
df['age'] = df.groupby(['pclass', 'sex'])['age'].transform(
```

```
    lambda x: x.fillna(x.median())
)
df['deck'] = df['deck'].cat.add_categories('Unknown')
df['deck'].fillna('Unknown', inplace=True)
df['embarked'].fillna(df['embarked'].mode()[0], inplace=True)
df['embark_town'].fillna(df['embark_town'].mode()[0], inplace=True)

df['sex'] = df['sex'].map({'male': 0, 'female': 1})
df = pd.get_dummies(
    df,
    columns=['embarked', 'class', 'who', 'deck', 'embark_town'],
    drop_first=True
)
df['family_size'] = df['sibsp'] + df['parch']
df.drop(['alive', 'adult_male', 'alone'], axis=1, inplace=True)
```

After these steps, df.isnull().sum() is zero for every column. The working frame has 23 columns: the target survived plus 22 predictors, of which 15 are Boolean dummy columns.

5.4 Model Training

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

X = df.drop('survived', axis=1)
y = df['survived']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
```

```
)


logreg = LogisticRegression()
logreg.fit(X_train, y_train)


rf = RandomForestClassifier()
rf.fit(X_train, y_train)
```

5.5 Evaluation Code

```
def plot_confusion_matrix(y_true, y_pred, model_name):
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(6, 4))
    sns.heatmap(
        cm, annot=True, fmt='d', cmap='Blues', cbar=False,
        xticklabels=['Died', 'Survived'],
        yticklabels=['Died', 'Survived']
    )
    plt.title(f'{model_name} Confusion Matrix', fontsize=16)
    plt.xlabel('Predicted Label', fontsize=12)
    plt.ylabel('True Label', fontsize=12)
    plt.show()


y_pred_logreg = logreg.predict(X_test)
print("Logistic Regression Accuracy:", accuracy_score(y_test, y_pred_logreg))
print(classification_report(y_test, y_pred_logreg))
plot_confusion_matrix(y_test, y_pred_logreg, "Logistic Regression")
```

```
y_pred_rf = rf.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))

print(classification_report(y_test, y_pred_rf))

plot_confusion_matrix(y_test, y_pred_rf, "Random Forest Regression")
```

5.6 Hyperparameter Tuning

```
from sklearn.model_selection import GridSearchCV



param_grid = {

    'n_estimators': [100, 200, 300],

    'max_depth': [None, 10, 20, 30],

    'min_samples_split': [2, 5, 10]

}

grid_search_rf = GridSearchCV(estimator=rf, param_grid=param_grid, cv=3)

grid_search_rf.fit(X_train, y_train)

print("Best parameters for Random Forest:", grid_search_rf.best_params_)



best_rf = RandomForestClassifier(

    max_depth=grid_search_rf.best_params_['max_depth'],

    min_samples_split=grid_search_rf.best_params_['min_samples_split'],

    n_estimators=grid_search_rf.best_params_['n_estimators']

)

best_rf.fit(X_train, y_train)

y_pred = best_rf.predict(X_test)

print(f'Accuracy of the optimized Random Forest: {best_rf.score(X_test, y_test):.4f}')

print(classification_report(y_test, y_pred))

plot_confusion_matrix(y_test, y_pred, "Random Forest Optimized")
```

5.7 Sample Output

Best parameters for Random Forest: {'max_depth': 20, 'min_samples_split': 5, 'n_estimators': 200}

Model	Accuracy	Died Precision / Recall / F1	Survived Precision / Recall / F1
Logistic Regression	0.8156 (81.56%)	0.83 / 0.86 / 0.85	0.79 / 0.76 / 0.77
Random Forest (default)	0.7821 (78.21%)	0.82 / 0.81 / 0.81	0.73 / 0.74 / 0.74
Random Forest (optimized)	0.8101 (81.01%)	0.81 / 0.88 / 0.84	0.80 / 0.72 / 0.76

Table 5.1: Test-set classification metrics (179 passengers; 105 died, 74 survived)

Model	TN (Died-Died)	FP	FN	TP (Survived-Survived)
Logistic Regression	90	15	18	56
Random Forest (default)	85	20	19	55
Random Forest (optimized)	92	13	21	53

Table 5.2: Confusion matrix counts reconstructed from the classification reports

Logistic Regression is the strongest of the three on this split. Tuning improved Random Forest from 78.21 percent to 81.01 percent by using 200 trees, a maximum depth of 20 and a minimum split of 5 samples. The optimized forest finds more true deaths (higher recall for class 0) but misses slightly more actual survivors than the logistic model. This is illustrative output of the notebook; actual digits can vary if the random forest seed changes, but the notebook used the split random_state=42.

CHAPTER 6: CONCLUSION

The Titanic Survival Prediction Using Machine Learning project successfully demonstrates a complete binary classification pipeline on the Seaborn Titanic dataset. Exploratory plots showed that class and gender are strongly related to survival. Missing values were treated, categorical fields were encoded, and a family_size feature was added. Logistic Regression and Random Forest were trained on an 80:20 split and judged with accuracy, precision, recall, F1-score and confusion matrices.

Logistic Regression reached 81.56 percent test accuracy. Default Random Forest reached 78.21 percent, and GridSearchCV improved it to 81.01 percent with max_depth=20, min_samples_split=5 and n_estimators=200. The study shows that careful preprocessing and a fair test split matter as much as the choice of algorithm, and that hyperparameter search can recover much of the gap between a simple forest and a linear baseline.

Key Outcomes

- Successful loading and cleaning of the 891-row Titanic dataset
- Exploratory analysis of survival against class, gender, age, fare, family and port
- Implementation of Logistic Regression and Random Forest in Scikit-learn
- Improvement of Random Forest through GridSearchCV
- Transparent comparison using classification reports and confusion matrices

Learning Outcomes

- Understanding of supervised classification and evaluation metrics
- Hands-on experience with Pandas, Seaborn and Scikit-learn
- Knowledge of missing-value imputation and categorical encoding
- Experience in train-test splitting and hyperparameter tuning

Future Scope

- Feature scaling and a tuned Logistic Regression or SVM baseline
- Cross-validated comparison of additional models such as Gradient Boosting or XGBoost
- Handling of class imbalance with class weights or resampling
- A simple web form that takes passenger attributes and returns a survival probability

CHAPTER 7: REFERENCES

[1] Scikit-learn Documentation, https://scikit-learn.org/stable/documentation.html

[2] Pandas Documentation, https://pandas.pydata.org/docs/

[3] Seaborn Documentation, https://seaborn.pydata.org

[4] Matplotlib Documentation, https://matplotlib.org/stable/contents.html

[5] Seaborn Titanic Dataset, https://github.com/mwaskom/seaborn-data

[6] Pedregosa et al., Scikit-learn: Machine Learning in Python, JMLR, 2011

[7] Machine Learning Study Material (College Notes)

[8] Kaggle Titanic: Machine Learning from Disaster, https://www.kaggle.com/c/titanic